

Internship Project Report

Dynamic Follow-up Questions for Attributing Failures of RAG Chatbots

By

Tanyarat Suksuwan
Kanyanat Wasuthawat
Sippakorn Kanin

Advisors

Asst. Prof. Dr. Thanapon Noraset
Mahidol University

Assoc. Prof. Dr. Natthawut Kertkeidkachorn
Japan Advanced Institute of Science and Technology (JAIST)

A Capstone Project Submitted in Partial Fulfillment of
the Requirements for

THE DEGREE OF BACHELOR OF SCIENCE
(INFORMATION AND COMMUNICATION TECHNOLOGY)

Faculty of Information and Communication Technology
Mahidol University
in collaboration with Japan Advanced Institute of Science and Technology

2026

Table of Contents

Abstract . . . 1

1 Introduction . . . 1
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Research Question and Objectives . . . 1
- 1.4 Scope . . . 2
- 1.5 Significance . . . 2
- 1.6 Structure of the Report . . . 2

2 Related Works . . . 2
- 2.1 Overview of the Review . . . 2
- 2.2 LLM-as-a-Judge . . . 2
- 2.3 Question Generation for Evaluation . . . 3
- 2.4 RAG Failure Types . . . 3
- 2.5 Research Gap . . . 4

3 Methodology . . . 4
- 3.1 Research Design . . . 4
- 3.2 Data Collection . . . 4
- 3.3 Judge Step . . . 4
- 3.4 Question Generation Methods . . . 4
- 3.5 Failure Prediction . . . 5
- 3.6 Human Annotation . . . 5
- 3.7 Evaluation Metrics . . . 6

4 Initial Results . . . 6
- 4.1 Findings from the Literature Review . . . 6
- 4.2 Reproduction of DragonBall Failure Classification . . . 6
- 4.3 Failure Rates on MTRAG . . . 6
- 4.4 Pilot Annotation Agreement . . . 7
- 4.5 Static Sub-Questions on a Multi-turn Conversation . . . 7

5 Discussion . . . 7

6 Timeline and Plan . . . 8

7 Conclusion . . . 8

References . . . 8

Appendices . . . 10

A Full Definitions of the Six Failure Types . . . 10

B Main Judge Criteria in LLM-as-a-Judge Studies . . . 11

C Types of Generated Questions . . . 11

D Example of Static Sub-Questions . . . 11

E RAG Systems Used . . . 12

F Candidate Datasets . . . 12

G Formulas for Cohen's Kappa . . . 12

H Draft Prompt for Dynamic Follow-up Question Generation . . . 13

Dynamic Follow-up Questions for Attributing Failures of RAG Chatbots

Tanyarat Suksuwan **Kanyanat Wasuthawat** **Sippakorn Kanin**
Faculty of Information and Communication Technology, Mahidol University
Advisors: Asst. Prof. Dr. Thanapon Noraset (Mahidol University)
Assoc. Prof. Dr. Natthawut Kertkeidkachorn (Japan Advanced Institute of Science and Technology)

Abstract

Retrieval-augmented generation (RAG) chatbots can give wrong answers for many reasons, such as missing knowledge, poor chunking, failed retrieval or weak grounding. Current evaluation methods can detect that an answer is wrong, but they cannot show which part of the RAG pipeline caused the error. They also focus mainly on single-turn questions, although real chatbots work in multi-turn conversations. In this work, we study whether dynamic follow-up questions, which are generated from how the RAG system actually answered, can attribute failures more accurately than static sub-questions. We propose a pipeline in which an LLM classifier assigns each failed turn to one of six failure types. We compare three inputs for the classifier: the failed conversation with no additional questions, the conversation with static sub-questions, and the conversation with dynamic follow-up questions. We evaluate each against human annotation using accuracy and F1 score. As a first step, we reproduce a single-turn failure classification baseline and obtain results close to the original study (39.0% vs. 40.3%). We also collect 885 failed turns from four RAG systems on the multi-turn MTRAG benchmark. A pilot annotation shows low agreement between annotators (mean Cohen's kappa 0.148), which suggests that RAG failure types are hard to separate, even for humans. These results establish the baseline and the dataset for the full comparison.

Keywords: retrieval-augmented generation, failure attribution, LLM-as-a-judge, question generation, multi-turn conversation

1 Introduction

1.1 Background

Large language models (LLMs) can answer many kinds of questions, but their knowledge is limited to their training data and they may produce statements that are not true. Retrieval-augmented generation (RAG) reduces this problem by connecting the LLM to an external knowledge base (Lewis et al., 2020). A RAG system has two main processes. In the **indexing process**, documents are split into chunks, each chunk is converted into a vector by an embedding model, and the vectors are stored in a database. In the **query process**, the user question is converted into a vector, the most similar chunks are retrieved, and the LLM uses these chunks to generate the answer. RAG is now widely used in chatbots for customer support, finance, government services and technical documentation.

1.2 Problem Statement

A RAG system has many stages, and an error in the final answer can come from any of them. Current RAG evaluation can often tell *that* an answer is wrong, but it does not clearly tell *why* it is wrong. For example, a wrong answer may happen because:

- the corpus does not contain the answer;
- the evidence was badly chunked;
- the retriever missed the correct chunk;
- the correct chunk was retrieved but not selected for generation; or
- the model answered only part of the question.

Without this information, developers do not know which component to improve. The problem is harder in multi-turn conversations, because a follow-up question often depends on earlier turns, and an error can come from the conversation history as well as from the current turn.

1.3 Research Question and Objectives

This project asks the following research question:

> *RQ: Can dynamic follow-up questions attribute failures of RAG more accurately than static sub-question generation?*

The objectives of the project are:

1. To develop and evaluate a **dynamic follow-up question** framework that uses runtime traces of

the RAG system (the conversation log, retrieved chunks and answers) and multi-turn context.
2. To attribute RAG pipeline failures (such as retrieval misses, context selection errors and grounding failures) more accurately than static sub-question generation methods.
3. To build a human-annotated dataset of failure types for multi-turn RAG conversations.

1.4 Scope

- The project covers both single-turn and **multi-turn** RAG conversations.
- The method should **generalize to many datasets**, so it is tested on a development set and a held-out set from different domains.
- The project focuses on the **English language** only.
- The project attributes failures after they happen; it does not aim to repair the RAG system.

1.5 Significance

If dynamic follow-up questions can identify the cause of a failure more accurately, RAG developers can find and fix the weak component faster, for example by changing the chunking strategy, the retriever or the prompt. The project also contributes a failure taxonomy that is extended to multi-turn conversations and a human-annotated set of failed multi-turn turns.

1.6 Structure of the Report

Section 2 reviews related work and states the research gap. Section 3 describes the methodology. Section 4 reports the initial results, and Section 5 discusses them. Section 6 gives the timeline and plan, and Section 7 concludes the report.

2 Related Works

2.1 Overview of the Review

We reviewed 63 papers on chatbot and RAG evaluation. The papers were recorded in a shared spreadsheet with their year, venue, link and labels (RAG-related, evaluation, LLM-as-a-judge, failure analysis, dataset). We found three main groups of related work (Table 1). A paper can belong to more than one group.

Research area	Papers	Main focus
LLM-as-a-judge	37	Using an LLM to score or compare answers
Question generation	16	Generating questions to test or probe a system
RAG failure attribution	11	Classifying why a RAG answer is wrong

Table 1: Number of reviewed papers in each research area.

2.2 LLM-as-a-Judge

LLM-as-a-judge means that a strong LLM is used to evaluate the output of another model, usually with a rubric and a scoring prompt. This approach is cheaper and faster than human evaluation, and several studies show that it agrees reasonably well with human preferences (Zheng et al., 2023; Li et al., 2024, 2025). In the RAG setting, frameworks such as RAGAS (Es et al., 2024) and ARES (Saad-Falcon et al., 2024) use LLM judges to measure faithfulness, answer relevance and context relevance.

Single-turn and multi-turn evaluation. Most LLM-as-a-judge papers evaluate single-turn question answering. In our review, only **18.2%** of the LLM-as-a-judge papers evaluate multi-turn or interactive conversations, while **81.8%** use single-turn settings. Examples of multi-turn work include MT-Bench (Zheng et al., 2023), MT-Bench-101 (Bai et al., 2024), RAD-Bench (Kuo et al., 2024), the FutureDial-RAG challenge, and simulated conversational QA with LLM-to-LLM interaction (Abbasiantaeb et al., 2024). MT-Bench-101 gives the judge the full conversation context, including earlier turns, and asks it to score the latest response with a rubric. Multi-turn evaluation is important because context shifts, dependencies between turns and error propagation over time can only be observed across several turns. Laban et al. (2025) also show that LLM performance drops clearly when a task is spread across several turns.

Judge criteria. The reviewed papers use many judge criteria. We grouped them into ten main criteria (Appendix B). The most common are helpfulness/relevance, factuality/faithfulness, citation support, reasoning, completeness, consistency, safety, abstention (negative rejection), robustness to judge bias, and efficiency/interpretability.

Summary. Most studies use single-turn evaluation, although most RAG chatbots are deployed in multi-turn settings. Also, judge criteria measure the correctness of the answer and its alignment with human expectations, but they do **not** attribute the root cause of a failure inside the RAG pipeline.

Method	Share	Description (examples)
Static single	50.0%	Questions are created once, before the system answers (RAGAS, ARES, RGB, HopWeaver)
Dynamic single	28.6%	One question is created from the system output or evidence (Self-RAG, JudgeLRM)
Dynamic follow-up	21.4%	A sequence of questions, each depending on the previous answer (LM-as-an-Examiner, RAD-Bench)

Table 2: Methods used to generate questions in the reviewed papers.

2.3 Question Generation for Evaluation

Types of generated questions. Question generation is often used to build test sets for RAG systems. The reviewed papers generate many types of questions, including fact (single-hop), multi-hop, information integration, negative rejection, noise robustness, counterfactual robustness, open-ended, multi-turn dialogue, fine-grained citation, domain-specific, knowledge-intensive and instruction-following questions (Appendix C). For example, the RGB benchmark (Chen et al., 2024) tests noise robustness, negative rejection, information integration and counterfactual robustness. However, these question types are mainly used to **evaluate** RAG performance, not to **probe** the cause of a RAG failure.

Methods of generating questions. We classified the question-generation papers by how the questions are created (Table 2).

Xie et al. (2024) is the closest work to this project. They break a question into **core**, **background** and **follow-up** sub-questions and check whether each sub-question is covered by the retrieved chunks and by the final answer. Their sub-question coverage agrees well with human preference and helps improve LLM-as-a-judge evaluation. However, their sub-questions are **static and single-turn**: they are created from the original question only, and they are not used to classify failure types.

Kim et al. (2024) propose *LLM-as-an-Interviewer*, where one LLM acts as an interviewer and asks another LLM follow-up questions based on its previous answers and on its error type. This dynamic approach can reveal more about a model's understanding and reduces problems such as data contamination and verbosity bias. It was designed for math and depth-QA tasks, not for RAG failure attribution.

Failure type	Part	Short definition
Knowledge Boundary	Retr.	The system does not recognize that the answer is outside the available evidence.
Chunking	Retr.	The information exists, but the chunk structure makes it hard to retrieve or use.
Retrieval	Retr.	Relevant evidence exists in the index but is not retrieved.
Context Selection	Retr.	Relevant evidence is retrieved but not selected for generation.
Grounding	Gen.	The answer is inconsistent with, not supported by, or goes beyond the evidence.
Response Coverage	Gen.	The answer is supported but incomplete for the user's request.

Table 3: Six RAG failure types used in this project (Retr. = retrieval part; Gen. = generation part).

Summary. Many types of questions exist, but they aim to evaluate RAG, not to probe RAG failures. The literature suggests two useful ideas: probing questions should depend on the conversation (**dynamic**, not static), and asking several questions may reveal more information (**follow-up**, not single).

2.4 RAG Failure Types

Barnett et al. (2024) describe seven failure points found when engineering RAG systems. RAGChecker (Ru et al., 2024) gives fine-grained diagnostic metrics for the retriever and the generator. Leung et al. (2025) propose a taxonomy of errors across the RAG stages (chunker, retrieval, reranker and generator), annotate failed responses on the DragonBall dataset, and introduce an automatic LLM-based error classifier called RAGEC. Their study uses single-turn questions and uses only the conversation trace, with no additional probing questions.

Based on these studies, this project uses **six failure types** (Table 3). The first four happen in the retrieval part of the pipeline, and the last two happen in the generation part. We found that these failure types can also be applied to multi-turn conversations if the definitions are extended (Appendix A).

2.5 Research Gap

The review shows the following gaps:

1. Most work uses LLM-as-a-judge to **evaluate** RAG answers, not to **attribute** the cause of failures.
2. A framework for failure-type classification exists, but only for **single-turn** RAG (Leung et al., 2025).
3. Probing questions have been used to investigate RAG behaviour (Xie et al., 2024), but they are **static** and are not conditioned on how the RAG system actually answered.

No study has tested whether **dynamic follow-up questions** can improve the attribution of RAG failures, especially in **multi-turn** conversations. This project addresses this gap.

3 Methodology

3.1 Research Design

This study uses an experimental design. The independent variable is the type of extra information given to a failure classifier: no probing questions (B0), static sub-questions (B1) or dynamic follow-up questions (B2). The dependent variable is how well the classifier's predicted failure type ($\hat{y}$) matches the human-annotated failure type (y). Figure 1 shows the overall pipeline.

3.2 Data Collection

Development set: DragonBall (single-turn). DragonBall is a single-turn question-answering dataset in finance, legal and medical domains, originally created in RAGEval (Zhu et al., 2024) and later annotated with failure types by Leung et al. (2025). It contains about 400 conversations answered by a standard RAG system (LLaMA-3 8B as the generator and GTE-Large English as the embedding model). We reproduced the retrieved chunks so that every case has the full input needed by our pipeline.

Development set: MTRAG (multi-turn). MTRAG is a human-created benchmark of multi-turn RAG conversations (Katsis et al., 2025). It contains 110 conversations with an average of 7.7 turns, for 842 evaluation tasks, in four domains: CLAPnq (Wikipedia), FiQA (finance), Govt (government) and Cloud (technical documentation). MTRAG has no existing failure-type labels. We therefore ran four RAG systems on it: a standard vector RAG, RAPTOR (Sarthi et al., 2024), Self-RAG (Asai et al., 2024) and HippoRAG 2 (Gutiérrez et al., 2025) (Appendix E). We then annotate the failure type of each failed turn.

Held-out set. To test generalization across domains, we will sample a small number of conversations from four to five other existing datasets, such as CORAL, MultiDoc2Dial and FinanceBench (Appendix F). The held-out set is not used when designing the prompts.

3.3 Judge Step

An LLM judge first decides whether each answer is correct by comparing it with the reference answer.

- **Single-turn:** one prompt judges the question-answer log. If the answer is correct, the pipeline stops for that case. If it is incorrect, the log is sent to question generation and failure prediction.
- **Multi-turn:** the judge evaluates the conversation turn by turn. If all turns are correct, the pipeline stops. If any turn is incorrect, every incorrect turn is sent to the next step. Therefore one multi-turn conversation can have several failure annotations.

3.4 Question Generation Methods

For every failed turn, we prepare three versions of the input to the failure classifier (Table 4).

Approach to dynamic follow-up questions. The dynamic follow-up method (B2) is adapted from LLM-as-an-Interviewer (Kim et al., 2024). In that work, the interviewer LLM receives the question, the model's answer history and the model's error type, and then writes a follow-up question that helps to find the source of the mistake. We make two changes for the RAG setting:

- Instead of giving the interviewer a known error type, we build **LLM-based signals** from the retrieved chunks, the gold chunk and the gold answer. For example, the interviewer can see whether the gold chunk was retrieved and whether the answer is consistent with the retrieved text.
- We use the "MATH Fail" prompt of Kim et al. (2024) as a template and replace its four math error types with our **six RAG failure types**.

The follow-up questions are then asked to the same RAG system, and its answers are added to the conversation log. The idea is that the way the system responds to a well-chosen follow-up question gives extra evidence about where the failure happened. For example, if the system answers correctly when the question is made more specific, the original

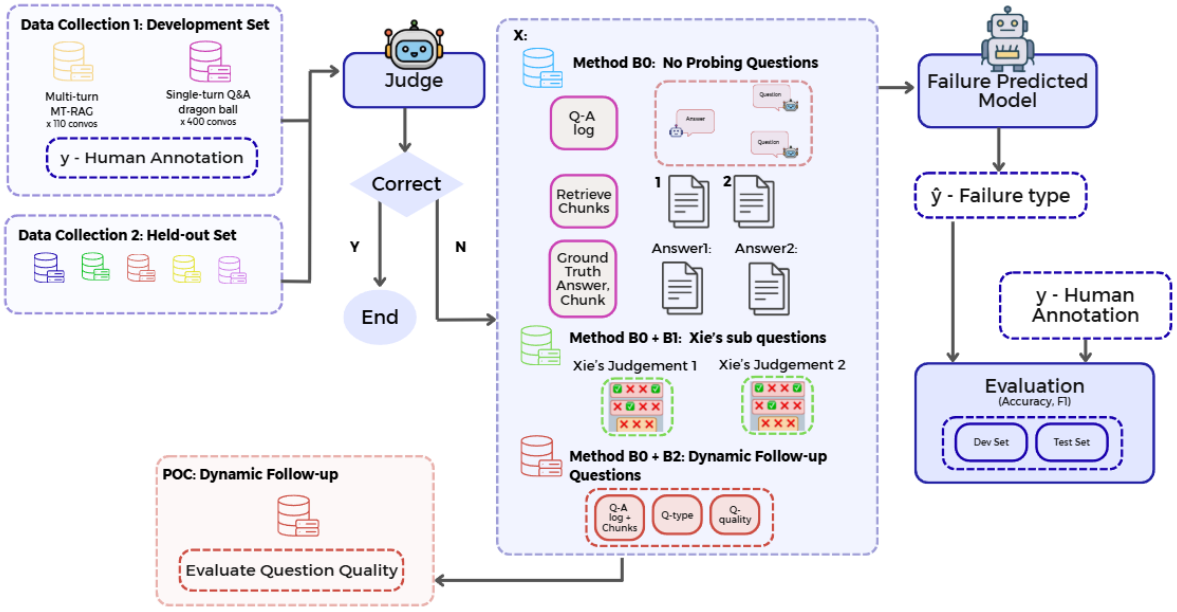


Figure 1: Overview of the research pipeline.

Method	**Input**	**Generation strategy**	**Output given to classifier** (X)
B0: No additional questions	Failed Q-A log; retrieved chunks; ground-truth answer and chunk	None (no additional questions).	Q-A log + chunks + ground truth
B0 + B1: Static sub-questions (Xie et al., 2024)	Same as B0	Split the original question into core, background and follow-up sub-questions. Check whether the retrieved text contains the facts for each sub-question, and whether the final answer includes them.	B0 + sub-question coverage of chunks + sub-question coverage of answer
B0 + B2: Dynamic follow-up (proposed)	Same as B0, plus the gold answer	Generate follow-up questions that depend on how the RAG system answered, and collect the RAG system's answers to them.	B0 + dynamic follow-up Q-A log

Table 4: The three question generation conditions.

failure is more likely to be a retrieval failure than a knowledge boundary failure. A draft prompt is given in Appendix H.

Question quality check (proof of concept). Before the full experiment, we will run a small proof of concept to check the quality of the generated follow-up questions. We will check whether each question is relevant to the failed turn, answerable from the corpus, and different from the original question. To support this check, we will also use judge criteria from the LLM-as-a-judge literature (Appendix B), such as relevance, clarity and consistency.

3.5 Failure Prediction

The failure prediction model is a zero-shot LLM classifier. Following DragonBall (Leung et al., 2025), we adapt their failure-classification prompts for our six failure types.

- **Input** (X)**:** the failed Q-A log, retrieved chunks, ground truth, and the question generation output (B0, B1 or B2).
- **Output** ($\hat{y}$)**:** one of the six failure types in Table 3.

The same LLM and the same prompt structure are used for all three conditions, so that any difference in performance comes from the added questions.

3.6 Human Annotation

Human labels (y) are the reference for evaluation. We wrote an annotation guideline with the defini-

tion, single-turn meaning and multi-turn meaning of each failure type, and built a web tool called *RAG Failure Annotation Studio*. For each failed turn, the tool shows the conversation, the retrieved documents, the documents used to generate the answer, the reference answer and the judge result. The annotator marks each failure type as *Pass*, *Fail* or *Not applicable* and can add notes and highlight evidence.

Three annotators label the same turns separately. We then compare their labels, discuss disagreements and update the guideline until the labels are consistent. Agreement is measured with pair-wise Cohen's kappa (Cohen, 1960):

$$\kappa = \frac{p_o - p_e}{1 - p_e} \quad (1)$$

where p_o is the observed agreement (the proportion of items where both annotators gave the same label) and p_e is the agreement expected by chance (Appendix G).

3.7 Evaluation Metrics

We compare $\hat{y}$ (failure type from the classifier) with y (failure type from humans) using **accuracy** and **F1 score** (macro-averaged over the six failure types). We report results separately for the development set and the held-out set, and for single-turn and multi-turn data. Finally, we compare the three conditions (B0 vs. B1 vs. B2). The research question is answered positively if B0+B2 gives higher accuracy and F1 than B0+B1.

4 Initial Results

This section reports the preliminary results obtained so far. The main comparison of B0, B1 and B2 has not yet been run; it is planned for October to December 2026 (Section 6).

4.1 Findings from the Literature Review

- Among the LLM-as-a-judge papers, 18.2% evaluate multi-turn or interactive settings and 81.8% evaluate single-turn settings.
- Among the question-generation papers, 50.0% use static single questions, 28.6% use dynamic single questions and 21.4% use dynamic follow-up questions.
- No reviewed paper uses dynamic follow-up questions to classify the failure type of a RAG system.

Metric	Count	%
Total cases	377	100.0
Judged correct	62	16.4
Possibly correct	15	4.0
Judged wrong	300	79.6
Failure type accuracy†	117 / 300	39.0
Failure stage accuracy†	173 / 300	57.7

Table 5: Results of the DragonBall reproduction. †Percentage of judged-wrong cases.

Dataset	RAG	Total	Fail	Rate
CLAPnq	Self-RAG	224	172	76.8%
CLAPnq	Vector	224	79	35.3%
Cloud	HippoRAG	205	120	58.5%
Cloud	Self-RAG	205	189	92.2%
Cloud	Vector	205	119	58.0%
FiQA	HippoRAG	199	132	66.3%
FiQA	RAPTOR	199	105	52.8%
FiQA	Self-RAG	199	159	79.9%
FiQA	Vector	199	101	50.8%
Govt	HippoRAG	214	117	54.7%
Govt	Self-RAG	214	182	85.0%
Govt	Vector	214	112	52.3%

Table 6: Turns marked as incorrect on MTRAG, by dataset and RAG system.

4.2 Reproduction of DragonBall Failure Classification

We reproduced the automatic error classification of Leung et al. (2025) on the DragonBall dataset (Table 5). Leung et al. (2025) report 40.3% accuracy for error type classification and 57.8% agreement with human annotation for stage classification. Our results (39.0% and 57.7%) are very close to these values.

4.3 Failure Rates on MTRAG

We ran four RAG systems on the MTRAG conversations and used the judge to mark each turn as correct or incorrect (Table 6). The standard vector RAG used a LLaMA-based generator and the Contriever-MS MARCO dense retriever.

Self-RAG had the highest fail rate in every domain (76.8% to 92.2%). The vector RAG on CLAPnq had the lowest fail rate (35.3%). We decided to exclude Self-RAG from the main annotation. Without Self-RAG, **885 failed turns** need annotation. The Self-RAG conversations are used to develop and test the annotation guideline.

Conversation	A1 vs A2	A1 vs A3	A2 vs A3
1	0.178	0.172	0.421
2	0.037	0.123	0.114
3	0.023	0.064	0.201
Mean	0.079	0.120	0.245

Table 7: Pair-wise Cohen's kappa in the pilot annotation (A = annotator).

4.4 Pilot Annotation Agreement

Three annotators labelled 18 failed turns from three Self-RAG conversations on CLAPnq using the B0 input. Table 7 shows the pair-wise Cohen's kappa for each conversation. Kappa values range from 0.023 to 0.421, and the mean over all pairs and conversations is 0.148. Following the scale of Landis and Koch (1977), this is *slight* agreement. Annotators 2 and 3 agreed more than the other pairs.

4.5 Static Sub-Questions on a Multi-turn Conversation

We applied the sub-question method of Xie et al. (2024) to a turn from a multi-turn conversation. The question was "who takes photos of planes in the air", and the RAG answer was "Aviation photographers". The method generated **20 sub-questions**: 1 core, 10 follow-up and 9 background. The core sub-question was "How do military photographers capture images of military aircraft in flight?". Many sub-questions covered topics far from the user's need, such as legal considerations, drone technology and safety (Appendix D).

5 Discussion

Reliability of the baseline pipeline. Our reproduction of the DragonBall classification gives almost the same accuracy as the original study (a difference of 1.3 percentage points for failure type and 0.1 for failure stage). This shows that our data processing, retrieved chunks and prompts are close to the original setup, so B0 is a fair baseline for later comparison. The results also confirm that zero-shot failure classification is difficult: even for single-turn data, the classifier chooses the correct failure type in only about four out of ten cases. Classifying the failure *stage* (retrieval or generation) is easier than classifying the exact failure *type*. This supports our idea that the classifier needs more evidence, and that extra probing questions may help.

Failure rates in multi-turn conversations. The MTRAG results show that multi-turn conversations are hard for all four RAG systems: even the best case fails on about one third of turns. This agrees with Katsis et al. (2025), who report that RAG systems struggle with later turns, unanswerable questions and context-dependent questions. The very high fail rate of Self-RAG (up to 92.2%) may be partly caused by a mismatch between its reflection-token output format and the multi-turn setting, and not only by weak retrieval or generation. Because such failures could add noise to the failure-type labels, we excluded Self-RAG from the main annotation. The high number of failures (885 turns) also means that there is enough data to compare B0, B1 and B2.

Low annotation agreement. The low kappa values in the pilot are an important finding. Possible reasons are:

- **Overlapping definitions.** Some failure types are close to each other, for example Retrieval and Context Selection, or Grounding and Response Coverage. One wrong answer can also have more than one cause.
- **Multi-turn context.** Annotators must decide whether the error comes from the current turn or from earlier turns, which adds judgement.
- **Small sample.** Only 18 turns were labelled, and kappa is unstable on small samples, especially when some labels are rare.
- **Early guideline.** The pilot used the first version of the guideline, before any discussion between annotators.

These results show that the guideline must be refined before the main annotation. We plan to add clear decision rules (for example, check the pipeline stages in a fixed order from Knowledge Boundary to Response Coverage), add worked examples, and allow a primary and a secondary failure type. Low human agreement also sets an upper limit for any automatic classifier, so improving it is necessary for a fair evaluation.

Limitations of static sub-questions. The example in Section 4 shows a weakness of static sub-questions. The method produced many sub-questions, but most were not related to the actual failure. The sub-questions are created from the question alone, before looking at how the RAG system answered, so they cannot focus on the part of the pipeline that failed. In a multi-turn conversation, they also do not consider earlier turns. Dynamic follow-up questions are designed to solve this problem, because each question depends on

Period	Activity
Sep 2026	Develop the dynamic follow-up question generation method.
Sep–Oct 2026	Collect human annotation: finalize the guideline; annotate 885 failed turns from MTRAG.
Oct–Nov 2026	Collect LLM failure predictions for B0, B1 and B2: single-turn DragonBall (Oct); multi-turn MTRAG and our annotated data (Nov).
Nov–Dec 2026	Improve the dynamic follow-up question generation.
Dec 2026	Compare and analyse the results.

Table 8: Project timeline.

the RAG system's answer and on signals from the retrieved and gold evidence. Whether this leads to more accurate failure attribution will be tested in the main experiment.

Limitations of the study.

- The main comparison of B0, B1 and B2 has not yet been completed, so the research question cannot be answered yet.
- The study uses an LLM judge to decide which turns are wrong. Judge errors can add false failures or miss real ones.
- Dynamic follow-up questions require extra calls to the RAG system and the LLM, which increases cost and time.
- The study covers English only and a limited number of RAG systems and domains.
- Human annotation is done by three student annotators, and agreement is currently low.

6 Timeline and Plan

Table 8 shows the plan for the remaining work.

7 Conclusion

This report presented a project on attributing the failures of RAG chatbots with dynamic follow-up questions. The review of 63 papers showed that most research uses LLM-as-a-judge to evaluate answers rather than to find the cause of failures, that most studies use single-turn settings, and that probing questions are usually static. Existing failure classification covers only single-turn RAG.

To address this gap, we designed a pipeline that compares three inputs for a zero-shot failure classifier: the failed conversation only (B0), static subquestions (B1), and dynamic follow-up questions (B2). We defined six failure types and extended their definitions to multi-turn conversations. The initial results show that our baseline reproduces the published DragonBall results closely, that multi-turn RAG systems fail often (35.3% to 92.2% of turns), and that 885 failed turns from MTRAG are available for annotation. The pilot annotation showed low agreement, so the guideline must be improved before the main annotation.

In the next months, we will develop and improve the dynamic follow-up question method, complete the human annotation, collect failure predictions for B0, B1 and B2, and compare the results with accuracy and F1 score. If dynamic follow-up questions improve failure attribution, the method can help developers find and fix weak components of RAG chatbots faster, especially in real multi-turn use.

References

Zahra Abbasiantaeb, Yifei Yuan, Evangelos Kanoulas, and Mohammad Aliannejadi. 2024. Let the LLMs talk: Simulating human-to-human conversational QA via zero-shot LLM-to-LLM interactions. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM '24)*.

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Ge Bai, Jie Liu, Xingyuan Bu, Yancheng He, Jiaheng Liu, Zhanhui Zhou, et al. 2024. MT-Bench-101: A fine-grained benchmark for evaluating large language models in multi-turn dialogues. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Brannelly, and Mohamed Abdelrazek. 2024. Seven failure points when engineering a retrieval augmented generation system. *arXiv preprint arXiv:2401.05856*.

Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. 2024. Benchmarking large language models in retrieval-augmented generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38.

Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46.

Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2024. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*.

Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From RAG to memory: Non-parametric continual learning for large language models. *arXiv preprint arXiv:2502.14802*.

Yannis Katsis, Sara Rosenthal, Kshitij Fadnis, Chulaka Gunasekara, Young-Suk Lee, Lucian Popa, Vraj Shah, Huaiyu Zhu, Danish Contractor, and Marina Danilevsky. 2025. MTRAG: A multi-turn conversational benchmark for evaluating retrieval-augmented generation systems. *arXiv preprint arXiv:2501.03468*.

Eunsu Kim, Juyoung Suk, Seungone Kim, Niklas Muennighoff, Dongkwan Kim, and Alice Oh. 2024. LLM-as-an-interviewer: Beyond static testing through dynamic LLM evaluation. *arXiv preprint arXiv:2412.10424*.

Tzu-Lin Kuo, Feng-Ting Liao, Mu-Wei Hsieh, Fu-Chieh Chang, Po-Chun Hsu, and Da-Shan Shiu. 2024. RAD-Bench: Evaluating large language models' capabilities in retrieval augmented dialogues. *arXiv preprint arXiv:2409.12558*.

Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. 2025. LLMs get lost in multi-turn conversation. *arXiv preprint arXiv:2505.06120*.

J. Richard Landis and Gary G. Koch. 1977. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174.

Kin Kwan Leung, Mouloud Belbahri, Yi Sui, Alex Labach, Xueying Zhang, Stephen Anthony Rose, and Jesse C. Cresswell. 2025. Classifying and addressing the diversity of errors in retrieval-augmented generation systems. In *Proceedings of EACL 2026*. ArXiv:2510.13975.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*.

Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, et al. 2025. From generation to judgment: Opportunities and challenges of LLM-as-a-judge. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, et al. 2024. LLMs-as-judges: A comprehensive survey on LLM-based evaluation methods. *arXiv preprint arXiv:2412.05579*.

Dongyu Ru, Lin Qiu, Xiangkun Hu, Tianhang Zhang, Peng Shi, Shuaichen Chang, et al. 2024. RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.

Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2024. ARES: An automated evaluation framework for retrieval-augmented generation systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Kaige Xie, Philippe Laban, Prafulla Kumar Choubey, Caiming Xiong, and Chien-Sheng Wu. 2024. Do RAG systems cover what matters? evaluating and optimizing responses with sub-question coverage. *arXiv preprint arXiv:2410.15531*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, et al. 2023. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Kunlun Zhu, Yifan Luo, Dingling Xu, Ruobing Wang, Shi Yu, Shuo Wang, et al. 2024. RAGEval: Scenario specific RAG evaluation dataset generation framework. *arXiv preprint arXiv:2408.01262*.

A Full Definitions of the Six Failure Types

Failure type	Description	Single-turn	Multi-turn
Knowledge Boundary	The system fails to recognize that the answer is outside the available evidence scope.	The corpus does not contain enough evidence, but the system answers anyway.	The system cannot tell whether the answer should come from retrieved evidence, prior conversation, or is unsupported by both.
Chunking	Relevant information exists in the source collection, but the chunk structure makes it hard to retrieve or use.	Needed evidence is split, incomplete, too broad, too narrow, or lacks surrounding context.	The evidence may span what the user has built up over several turns; no chunk holds a self-contained answer to the (often under-specified) follow-up question, or sub-answers across turns are scattered.
Retrieval	The system fails to retrieve relevant evidence that exists in the indexed corpus.	A clear standalone query has relevant chunks available, but they are not retrieved.	The follow-up query is underspecified or context-dependent, causing retrieval for the wrong entity, topic or aspect.
Context Selection	Relevant evidence is available after retrieval but is not selected for generation.	Relevant chunks are retrieved but dropped, ranked too low, or replaced by distractors.	The system fails to select the right evidence across turns, especially when the answer needs both current-turn evidence and earlier context.
Grounding	The answer is inconsistent with, unsupported by, or overextends beyond the selected evidence.	The model fabricates, contradicts evidence, or makes claims not supported by the context.	The model contradicts retrieved evidence or the established conversation state, mixes entities across turns, or imports unsupported prior assumptions.
Response Coverage	The answer is supported but incomplete relative to the user's request.	The answer omits part of a multi-part question, misses key evidence, or gives only a partial answer.	The answer addresses the latest turn but misses accumulated constraints, comparisons, requested scope, or unresolved pieces from earlier turns.

Table 9: Full definitions of the six failure types for single-turn and multi-turn conversations.

Table 9 gives the full definition of each failure type, with its meaning in single-turn and multi-turn conversations.

B Main Judge Criteria in LLM-as-a-Judge Studies

Criteria	What it measures
Helpfulness / relevance	Does the answer solve the user's question or task?
Factuality / faithfulness	Is the answer correct and grounded in the given context?
Citation / evidence support	Does it use and cite the provided evidence correctly?
Reasoning / logic	Can it explain steps, handle multi-hop logic and avoid weak reasoning?
Completeness / coverage	Does it include all important information or atomic facts?
Consistency / coherence	Is the answer stable, clear and not contradictory?
Safety / harmlessness	Does it avoid harmful, unsafe or policy-violating content?
Abstain / negative rejection	Can it say "not enough information" when needed?
Robustness / bias sensitivity	Is the judge resistant to position bias, verbosity, fake citations and style artifacts?
Efficiency / interpretability	Is the judgment transparent and not overthinking simple tasks?

Table 10: Main judge criteria found in the review.

C Types of Generated Questions

Type	Description
Fact (single-hop)	The answer is found in exactly one sentence or document.
Multi-hop / cross-document	The model must connect clues spread across several documents.
Information integration	Separate facts from several sources are merged into one answer.
Negative rejection	The model should refuse when the retrieved text does not contain the answer, instead of hallucinating.
Noise robustness	Tests how well the model ignores irrelevant text to find the true answer.
Counterfactual robustness	Checks whether the model trusts wrong information in the context.
Open-ended	Broad questions that need long, natural-language answers.
Multi-turn dialogue	A back-and-forth conversation where queries change based on history.
Fine-grained citation	The system must show exactly which line or document it used.
Domain-specific	Questions on specialized, technical documents (e.g. medical or product QA).
Knowledge-intensive	Large academic datasets that test open-domain knowledge (e.g. KILT, SuperGLUE).
Instruction following	Grades how well models follow format and style constraints.

Table 11: Types of generated questions in the literature.

D Example of Static Sub-Questions

Question: who takes photos of planes in the air. **RAG answer:** Aviation photographers. **Number of sub-questions:** 20 (core: 1; follow-up: 10; background: 9). **Core sub-question:** How do military photographers capture images of military aircraft in flight? Examples are listed below (T = covered, F = not covered).

	Sub-question
T	What are the different types of aviation photography?
F	Who are professional aviation photographers?
T	What equipment is used by photographers who take photos of planes in the air?
T	What techniques do photographers use to capture images of planes in flight?
F	Are there specific companies that specialize in aerial photography of aircraft?
F	Do airlines employ photographers to take pictures of their aircraft?
T	How do military photographers capture images of military aircraft in flight?
F	What role do hobbyists play in aviation photography?
F	What are the legal considerations for taking photographs of planes in the air?
F	How do photographers gain access to airfields or secure areas for shooting?
T	What events or airshows are popular for photographing planes in the air?
F	How do photographers ensure safety while taking photos of planes in the air?
T	What are the challenges faced by photographers when shooting planes in the air?
F	How has drone technology impacted the photography of planes in the air?

Table 12: Examples of static sub-questions (Xie et al., 2024).

E RAG Systems Used

RAG system	Main idea
Vector RAG	One-step retrieve-and-generate with a dense retriever; no reasoning step (Lewis et al., 2020).
RAPTOR	Builds a tree of recursive summaries and retrieves across several levels of abstraction (Sarthi et al., 2024).
Self-RAG	Uses reflection tokens to decide when to retrieve and to critique its own output (Asai et al., 2024).
HippoRAG 2	Uses a knowledge graph and Personalized PageRank for multi-hop associative retrieval (Gutiérrez et al., 2025).

Table 13: RAG systems used to collect MTRAG conversations.

F Candidate Datasets

Dataset	Task	Domain / notes
DragonBall	Single	Finance, legal, medical; human failure-type labels (development set).
MTRAG	Multi	Wikipedia, finance, government, cloud; 110 conversations, 842 tasks (development set).
CORAL	Multi	Open-domain; knowledge-intensive, topic shift, citation labels.
MultiDoc2Dial	Multi	Open-domain; dialogues grounded in multiple documents.
FinanceBench	Single	Finance; questions on company financial documents.
RAGBench	Single	Multiple domains; candidate for held-out set.
KILT-HotpotQA	Multi-hop	Open domain; candidate for held-out set.
KILT-WoW	Single	Open domain; knowledge-intensive, free-form responses.

Table 14: Candidate datasets for the development and held-out sets.

G Formulas for Cohen's Kappa

$$p_o = \frac{a}{n} \tag{2}$$

$$p_e = \sum_l \frac{r_l}{n} \times \frac{c_l}{n} \tag{3}$$

$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{4}$$

where a is the number of items where both annotators gave the same label, r_l and c_l are the numbers of items that the first and second annotator gave label l, and n is the number of items. Interpretation (Landis and Koch, 1977): below 0 = poor; 0.00–0.20 = slight; 0.21–0.40 = fair; 0.41–0.60 = moderate; 0.61–0.80 = substantial; 0.81–1.00 = almost perfect.

H Draft Prompt for Dynamic Follow-up Question Generation

This is a working draft adapted from the "MATH Fail" prompt of Kim et al. (2024). It will be revised during the project.

```
Task: You are a question generator for diagnosing a retrieval-augmented generation (RAG) chatbot.
Below is a conversation in which the chatbot gave an incorrect answer, together with the retrieved
chunks and diagnostic signals. Your role is to write one follow-up question that will help reveal
which part of the RAG pipeline caused the failure.

Instructions:
1. Read the diagnostic signals (for example: whether the gold chunk was retrieved; whether the gold
chunk was used in the answer; whether the answer is supported by the retrieved text). Do not disclose
the signals or the correct answer to the chatbot.
2. Consider the six possible failure types: Knowledge Boundary, Chunking, Retrieval, Context
Selection, Grounding, Response Coverage.
3. Write a follow-up question that separates the most likely failure types. For example, rephrase the
question with a clearer entity to test Retrieval, ask for the source passage to test Grounding, or ask
about the missing part to test Response Coverage.
4. The question must fit naturally in the conversation and must be answerable from the corpus if the
system works correctly.

Input data: Conversation history: {dialogue_history} | Current question: {question} | RAG answer:
{rag_answer} | Retrieved chunks: {retrieved_chunks} | Gold answer (do not disclose): {gold_answer} |
Gold chunk (do not disclose): {gold_chunk} | Diagnostic signals: {signals}

Response format (JSON): {"target_failure_types": [...], "follow_up_question": "..."}
```